Unit I :: Document Structuring and Formatting in LaTeX

SEC-03-PHY-91-R-03

Dr. Parag Bhattacharya
Department of Physics
Rangapara College (Autonomous)

Syllabus

Unit I: Document Structuring and Formatting in LaTeX

Overview of markup-based vs. WYSIWYG systems, advantages of separating content from presentation; structure of a LaTeX document: preamble and body; plain text: whitespaces, newline, hyphens and dashes, quotation marks, special characters; rich text formatting: italics, boldface, underline and emphasis; creating lists: itemized lists, enumerated lists, nested lists; comments; labels and referencing.

1 An overview of document preparation systems

A *document preparation system* is any software that is used to *create*, *edit*, *format* and *structure* text for printing or digital viewing. Rather than just acting as digital typewriters, these systems manage the entire lifecycle of a document.

There are four main functions of a document preparation system:

a) *Content ingestion*, which means providing an interface via which the user can input and edit text.

b) *Structural tagging*, which refers to defining what the text is, e.g. chapter title, section title, footnote or a caption.

c) *Stylistic rendering*, which indicates the application of typefaces, margins, line spacing and alignment of structural elements.

d) *Output compilation*, which signifies the conversion of the working file into a printed sheet, or a ubiquitous format like PDF, HTML or EPUB.

1.1 Categories of document preparation systems

Document preparation systems can be broadly classified into two categories:

1. **Word processing software:** Such software are designed for general-purpose writing, editing, and basic layout of everyday documents like letters, reports and memos.
    - Typically, the interface is WYSIWYG which stands for *What You See Is What You Get*. This means that text and formatting appear on the screen as they will approximately look when printed.
    - Elements like images and tables are usually anchored directly to the text flow and move down the page as you type or add content.
    - The formatting approach relies heavily on direct manual formatting (such as changing font sizes, italicising or spacing on the fly) or applying predefined template styles.
    - Some examples are Microsoft Word, Google Docs and LibreOffice Writer.
2. **Typesetting software:** Such software are designed for publication-grade document composition, strict typographic control (such as kerning, hyphenation, and line-breaking algorithms) and complex scientific or multi-column layouts.
    - The interface is often markup-based. Thie means that writing plain text mixed with structural code or commands that get compiled into a final PDF, rather than a direct visual editor.
    - Automated algorithms are used to manage images and tables, page breaks, mathematical formulae and bibliographic references without manual nudging.
    - The formatting approach strictly separates content from presentation, i.e the user focuses on the content and its semantics (e.g., heading, theorem, citation etc.), while the underlying algorithm applies exact, consistent formatting rules.
    - Some examples are LaTeX, Typst and Troff.

1.2 Separation of content and formatting in scientific documentation

In scientific documentation, enabling the separation of content (e.g. text, data, structure) and formatting (e.g. font, margins, layout) allows the author to shift one's focus from presentation to substance. Such an approach is provided by LaTeX, with the following advantages:

a) Authors write in plain text using semantic tags to define structural elements, while the software handles the visual rendering in the background. Consequently, the author can spend their time refining arguments, data accuracy and prose, rather than struggling with erratic margins, font consistency or manual line breaks.

b) Global style files control the entire document's appearance. So any formatting changes instantly updates the entire manuscript consistently, thereby minimising human error.

c) Scientific documents rely heavily on interconnected data. When content is decoupled from style, the system can dynamically manage complex modules without manual intervention.

   i. Mathematical equations are rendered using precise algorithmic typesetting that maintain correct spacing and symbol scales regardless of surrounding text adjustments.

   ii. Citations, figures, tables and equations are linked to logical labels. If a figure is moved or added, the system automatically renumbers every reference throughout the document.

   iii. Bibliographies are generated directly from a central database file. Thereby the system parses the database and formats the citations perfectly according to the required style guide.

d) Submitting a paper to a different journal often requires an entirely new layout. With a separated system, authors do not need to manually reformat headers, margins or citation styles. They simply swap out the publisher's template file or package declaration, and the document automatically reshapes itself to meet the new submission guidelines.

e) Plain-text content files are exceptionally lightweight and compatible with standard version control systems like Git. Unlike proprietary binary files such as DOCX, plain-text documents allow co-authors to view precise line-by-line changes, merge collaborative edits seamlessly and track the manuscript's evolution without the risk of file corruption.

1.3 What is LaTeX?

LaTeX is free, open source, markup-based software for typesetting documents. In other words, it is a document preparation system. LaTeX was initially developed in the 1980's by Leslie Lamport, based on Donald Knuth's TeX typesetting engine.

1.3.1 Benefits of LaTeX

LaTeX is best suited for scientific and technical documents, because of its superior typesetting of mathematical formulae, consistency of high-quality output, and ease of handling large and complex documents. Some of the benefits of LaTeX are listed below:

1. Ability for automatic numbering of equations, tables, figures and sections.

2. Capabilities for cross-referencing labels.
3. Automatic generation of lists of contents, figures, tables, indexes, glossaries and bibliography.
4. Versatility of document types that can be prepared, such as articles, reports, letters, books and presentation slides.
5. Flexibility in terms of availability of various templates for a given type of document.
6. Portability in terms of source files being compatible across systems and output being device-independent.
7. Scalability in terms of handling documents of any length without performance degradation.
8. Support from a massive online community of other LaTeX users from all over the world.

1.3.2 Working with LaTeX

There are two ways of working with LaTeX:

A. Installing the LaTeX compiler and an optional IDE on one's own computer.
   - Examples of LaTeX compilers are MikTeX and TeXLive.
   - Examples of IDEs are TeXstudio and TeXmaker.
   - *Note*: Even though installing LaTeX is recommended, it may take up about 8 GB space on the hard drive and at least 2 hours for the installation.

B. Using LaTeX online *in the cloud*, with the minimum requirement being a browser on an internet-connected device.
   - Examples of online LaTeX services are Overleaf and Inscrive.
   - *Note*: Even though Overleaf is free to use, it has an upper limit on the compilation time, and this limit can be raised with a paid subscription. So for a free account, with an increase in the complexity of the document, the chances of the document not compiling also increases.

1.4 Working with LaTeX on Overleaf

Overleaf is an excellent starting point for someone who is getting started with using LaTeX without getting overwhelmed with the hassle of installation.

Some of the benefits of Overleaf are as follows:

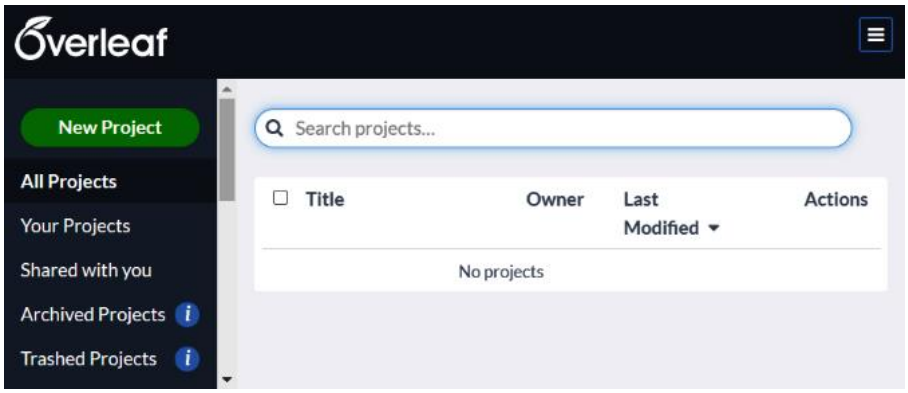
- It can be accessed on any device, such as PC, laptop, tablet or smartphone.
- It can be accessed from a locked-in work computer, where one is not allowed to install any software.

- One can access one's files on any device simply by logging in with one's password.
- One can collaborate with others to work on the same project.
- It provides a real-time view of the PDF output.
- It allows one to annotate the LaTeX code with comments and replies.

1.4.1 Creating a LaTeX project online

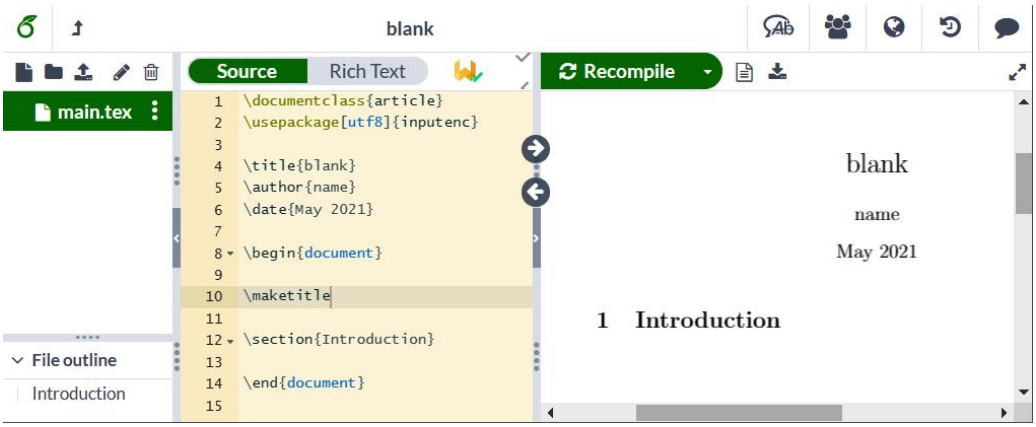
I. For a one-time registration, either click on `Register` on the Overleaf home page or visit `https://www.overleaf.com/register`. Enter an email address and choose a password.

II. To log in to Overleaf, either click on `Login` on the Overleaf home page or visit `https://www.overleaf.com/login`.

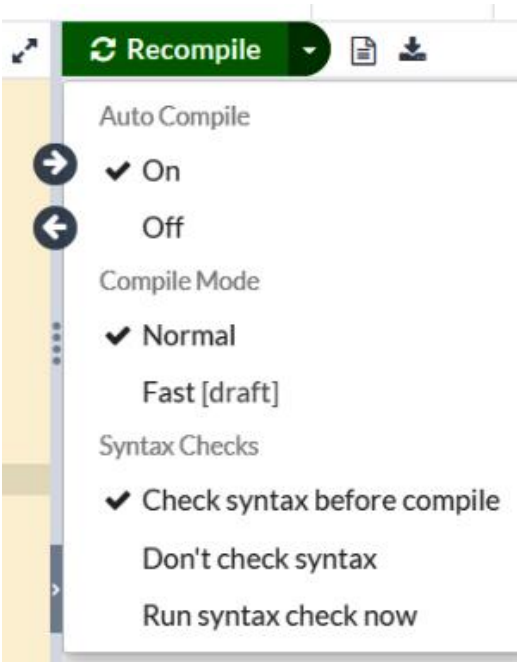


III. To create a new project, click the `New Project` button. A drop-down list will appear, where one can choose to have a blank project or one based on a template. For now, we just choose `Blank Project`.

IV. We enter a name for the project and Overleaf creates one with the bare minimum of code.



V. When any new changes are made to the project, we can view their effect on the output PDF by clicking on the `Recompile` button or pressing `Ctrl+S`. Enabling the `Auto Compile` option in the drop-down menu will ensure that the document recompiles automatically while one is typing.



2 Working with a LaTeX document

2.1 Building blocks of a LaTeX document

In LaTeX, *macros* (also called *commands*) and *environments* are the two fundamental building blocks used to define the structure, styling and behaviour of a document.

2.1.1 Macros in LaTeX

A *macro* is a single instruction given to the LaTeX compiler to perform a specific action, insert content or change a setting. All macros begin with a backslash (\), followed by a descriptive name consisting of letters or a single special character, for example `\section`.

Macros can accept *arguments* to control their behaviour. There are two types of arguments that can be passed to a macro:

1. *Mandatory arguments* are enclosed in braces { }. The macro will fail or behave incorrectly if these are omitted.
2. *Optional arguments* are enclosed in brackets [ ]. If omitted, LaTeX uses a default value.

In general, we have the following three variations in the syntax of macros:

```
\macro
\macro{mandatory arguments}
\macro[optional arguments]{mandatory arguments}
```

where the mandatory and optional arguments are separated by commas. For example, in the macro `\documentclass[12pt,a4paper]{article}`, `article` is a mandatory argument while `12pt` and `a4paper` are optional arguments.

2.1.2 Environments in LaTeX

An *environment* is a delimited block of code that applies specific structural rules, formatting, or layout settings to a specific region of content. Every environment starts with `\begin{name}` macro and ends with a matching `\end{name}` macro. Both macros require the exact same environment name as their argument.

Environments establish a *local scope*. Any formatting changes, counters or local declarations activated within an environment are automatically cancelled when the `\end{name}` macro is reached.

Environments can also accept mandatory { } and optional [ ] arguments immediately following the `\begin{name}` macro.

In general, we have the following syntax for an environment:

```
\begin{environment}[optional arguments]{mandatory arguments}
  <Content affected by the environment>
\end{environment}
```

where again the mandatory and optional arguments are separated by commas. For example, in the following environment which creates a table

```
\begin{tabular}[t]{ccc}
  abc & xy & def \\
  xyz & mn & pqr
\end{tabular}
```

`ccc` is a mandatory argument that creates three centre-aligned columns and `t` is an optional argument that specifies the vertical alignment of the table.

2.2 Structure of a LaTeX document

The simplest LaTeX document must contain the bare minimum code, as shown below. The text `Hello, world!` can be replaced with any other text.

```
\documentclass{article}
\title{My First Document}
\author{John Doe}
\date{26 Oct. 2026}

\begin{document}
  \maketitle

  \section{Introduction}
    Here is my first attempt in creating a document using \LaTeX.
\end{document}
```

2.2.1 The preamble

Every LaTeX document is an instance of a template known as a *class*, which provides customizable features. There are classes for books, journal articles, letters, presentations, posters, and many more. With the macro in the first line `\documentclass{article}`, we have chosen the `article` class, which provides a standard LaTeX template for smaller document.

We can also specify document properties using macros such as `\title`, `\author`, `\date` and any other general document-wide definitions.

This part of the document is known as the **preamble**.

```
\documentclass{article}
\title{My First Document}
\author{John Doe}
\date{26 Oct. 2026}
```

2.2.2 The document body

The \begin{document} macro demarcates the end of the preamble and the beginning of the actual document, while the \end{document} macro marks its end. Everything within this document environment appears in the document output. Here we create a title page using the \maketitle macro, a section titled Introduction using the \section macro and some text.

This part of the document contained within the document environment is known as the **document body**.

```
\begin{document}
  \maketitle

  \section{Introduction}
    Here is my first attempt in creating a document using \LaTeX.
\end{document}
```

Everything that follows the document body is ignored by LaTeX.

3 Plain text formatting

Insert section content.

4 Rich text formatting

Insert section content.

5 Creating lists

Insert section content.

6 Including comments

Insert section content.

7 Labels and cross-referencing

Insert section content.